

54. Écrire une musique

Une mélodie qui joue toute seule, pendant que le jeu tourne.

1. Ce qu'il faut comprendre

Un `Air` s'écrit comme une tuile se dessine : une suite de pas, dans le programme. Chaque pas est une note et son volume — `"D04 12"` —, ou l'un des deux pas qui n'en sont pas : `"--"` fait taire la voix, `"=="` laisse la note d'avant continuer.

C'est `"=="` qui fait les notes longues. Sans lui, une blanche s'écrit en jouant la même note à chaque pas, et l'oreille entendrait quatre coups au lieu d'une note tenue.

`jouer(voix, AIR, vitesse)` le lance. La **vitesse** est le nombre d'images que dure un pas : huit images font un peu moins de huit pas par seconde. Un quatrième argument, `1`, le fait recommencer sans fin.

Ensuite, **le programme n'a plus rien à tenir**. L'air avance tout seul, soixante fois par seconde, dans l'interruption de la console — même si le jeu, lui, est en retard. Un tempo qui ralentirait avec la charge du jeu ne serait pas un tempo.

La **partition ci-dessous** écrit ces lignes à la souris, et le bouton « Écouter » la fait entendre sans même compiler. Les deux voix chantantes sont la 1 et la 2 ; la quatrième, celle du bruit, se frappe avec `bruit(6, 10)`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Air>` → [53.1. Air — écrire une musique](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <jouer>` → [53.2. La fonction jouer\(\) — jouer un air tout seul](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <silence>` → [52.2. La fonction silence\(\) — faire taire une voix](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1160 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	74 o	← la console, à l'allumage — la mise en route de la console
main()	169 o	← démarrage — ta fonction
VBlank	11 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AvancerAirs	308 o	← VBlank — fait avancer la musique à chaque image
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	76 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	192 o	← CopierTuiles — 12 lettres sur 43

```

Notes          120 o ← AvancerAirs  – la table des notes de musique
"LA MUSIQUE"    10 o ← texte()  – un texte gravé
"A ARRETER OU J..." 19 o ← texte() – un texte gravé
THEME_0         32 o ← jouer()  – une table gravée
BASSE_1         32 o ← jouer()  – une table gravée

```

Lettres gravées : AEIJLMOQRSTU

Pas gravé (jamais appelé) : AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert

Fonctions du langage appelées :

```

jouer()   × 2   106 o sur place, dans main() + THEME_0 (32 o), BASSE_1 (32 o)
texte()   × 2    22 o sur place, dans main() + EcrireTexte (10 o)
silence() × 2    12 o sur place, dans main()
bouton()  × 1     5 o sur place, dans main() + LireManette (37 o)
image()   × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <Air>      // un air de musique, note par note
#include <texte>    // écrit un texte à l'écran
#include <jouer>    // joue un air tout seul
#include <bouton>   // lit un bouton de la manette
#include <silence>  // fait taire une voix

```

2. UNE MÉLODIE — THEME

```

Air THEME = {
    "D04 12", "=", "MI4 12", "=", "SOL4 12", "=", "D05 13", "=",
    "SI4 11", "=", "SOL4 11", "=", "MI4 11", "=", "--", "--",
};

```

3. UNE MÉLODIE — BASSE

```

Air BASSE = {
    "D03 8", "=", "=", "=", "SOL3 8", "=", "=", "=",
    "MI3 8", "=", "=", "=", "--", "--", "--", "--",
};

```

4. UNE VARIABLE — JOUE

```
uint8_t joue = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    texte(4, 5, "LA MUSIQUE");
    texte(1, 8, "A ARRETER OU JOUER");

    jouer(1, THEME, 8, 1);
    jouer(2, BASSE, 8, 1);
    joue = 1;

    while (true) {
        image();

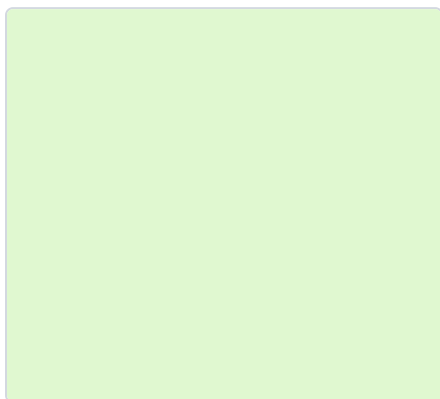
        if (bouton(A) && joue) {
            silence(1);
            silence(2);
            joue = 0;
        }
    }

    return 0;
}

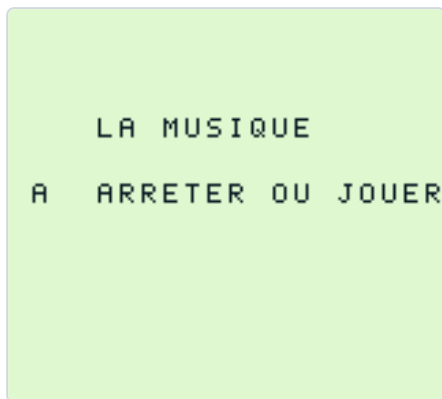
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



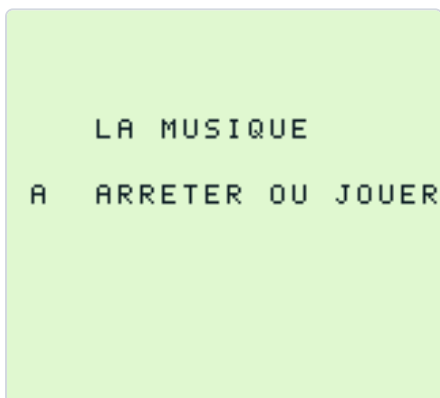
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé

5. Ce qu'on doit voir

Rien de neuf à l'écran — mais deux voix qui jouent, et « A » qui les arrête.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre est écrit
- ✓ la voix 1 joue le DO4 du début (329, 392, 524, 495, 262 Hz)
- ✓ puis le MI4 et le SOL4 — l'air avance tout seul
- ✓ la voix 2 tient la basse (131, 196, 165 Hz)

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

THEME

BASSE

+ air

MIDI

WAV

THEME

▶ Écouter

♪ note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas

`jouer(1, THEME, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.